

Eye Tracking

Table of Contents

1	Setup	1
2	The Algorithm – Used to Label Data	2
3	CNN Overview	9
4	Data Preparation Augmentation	10
5	CNN Training	10
6	Inference	12
7	Additional Notes	12

1 Setup

Required

- `opencv`
- `tensorflow`
- `onnxruntime`
- `haarcascade_eye.xml`

2 The Algorithm – Used to Label Data

1. Coarse Localization

Using OpenCV's pretrained Haar Cascade classifiers ([link](#)), the frame is cropped to focus on the eye region. This reduces the search space and significantly lowers complexity.

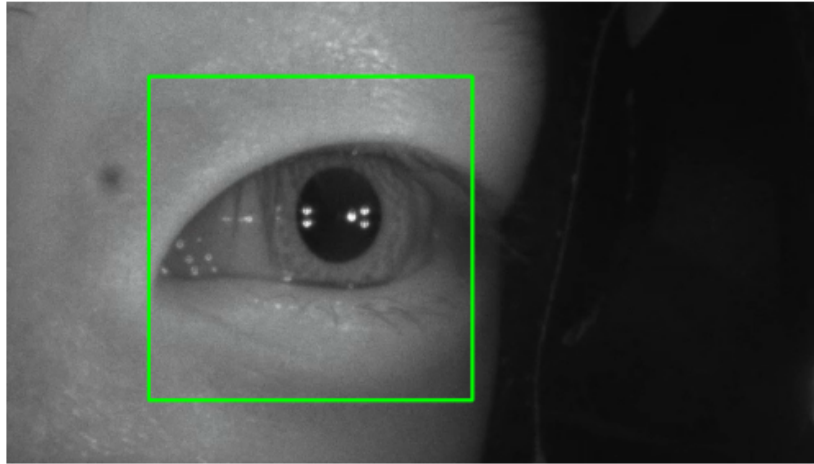


Figure 1: Eye ROI

2. Preprocessing

- Crop the eye region
- Remove bright glints via thresholding
- Convert to grayscale
- Apply Gaussian blur to suppress noise

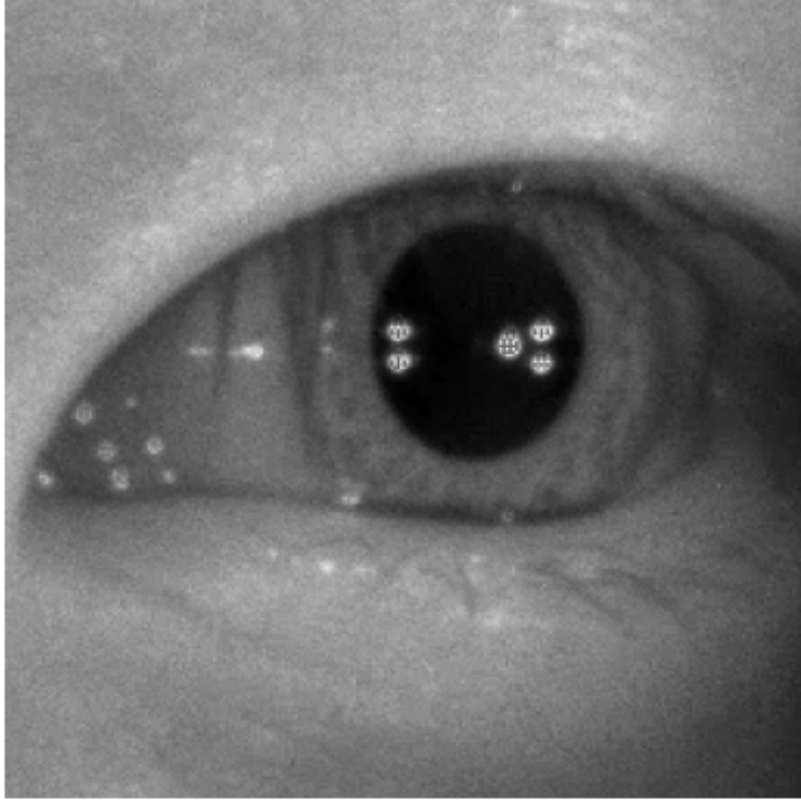


Figure 2: Eye Crop

3. Pupil Color Estimation

Assuming the pupil is the darkest region in the cropped eye image, we perform a grid search to locate the darkest square region and use its average intensity as a threshold baseline for binarization.

Grid Search Calculation:

Let the image be divided into $N \times N$ grid cells, and let $G_{i,j}$ be the set of pixel intensities in the cell at row i , column j . Then, the mean intensity $\mu_{i,j}$ of each cell is:

$$\mu_{i,j} = \frac{1}{|G_{i,j}|} \sum_{(x,y) \in G_{i,j}} I(x,y)$$

The **darkest region** is found by:

$$(i^*, j^*) = \arg \min_{i,j} \mu_{i,j}$$

The value μ_{i^*, j^*} becomes the **dark baseline threshold** for later steps.

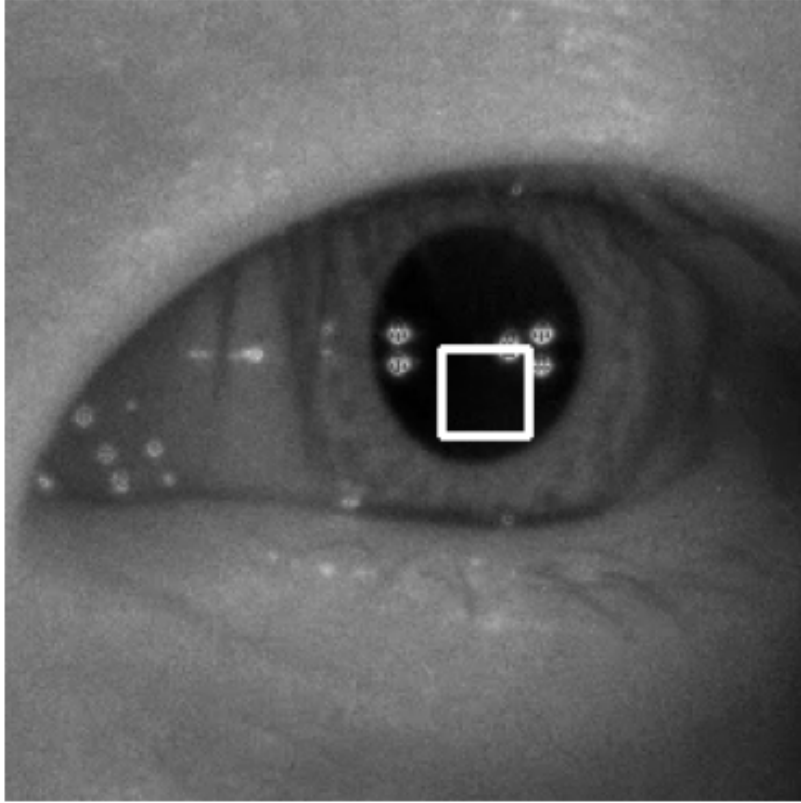


Figure 3: Eye crop

4. Multi-Threshold Segmentation

The image is thresholded at multiple values offset from the dark baseline. This helps handle lighting variability and different pupil contrasts across frames. Each thresholded image highlights possible pupil candidates.

Binary Thresholding Function:

Let $I(x, y)$ be the grayscale intensity at pixel (x, y) , and T be the threshold value (e.g., dark baseline + 21). Then the binarized image $B(x, y)$ is:

$$B(x, y) = \begin{cases} 255 & \text{if } I(x, y) < T \\ 0 & \text{otherwise} \end{cases}$$

Example: Image thresholded at baseline + 21



Figure 4: Eye crop output

5. Contour Extraction

Contours are extracted from each thresholded image. Small contours and those touching the image edge are discarded. The result is a point cloud approximating the pupil boundary.

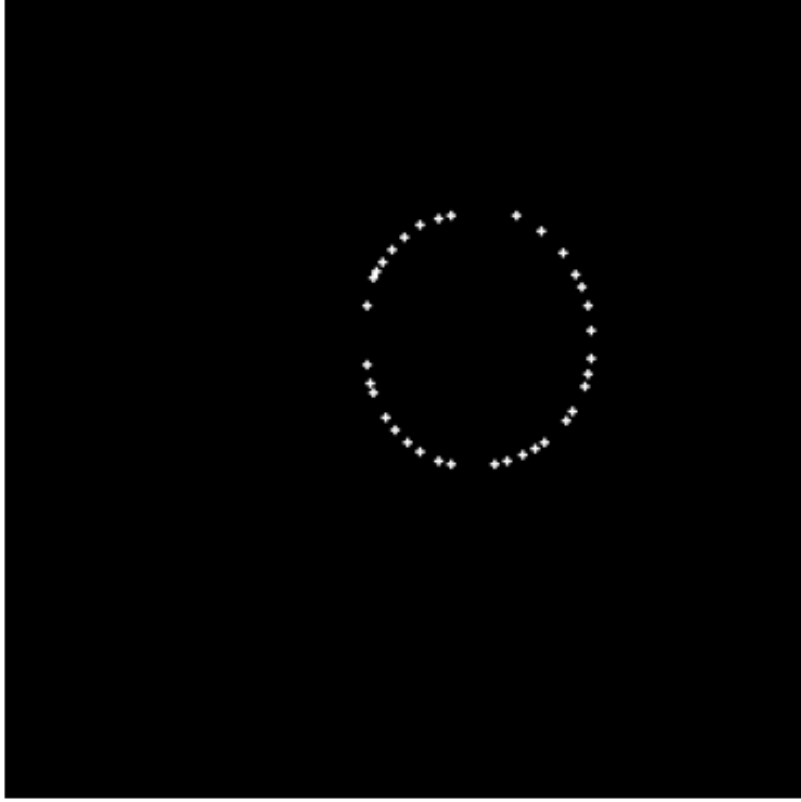


Figure 5: Contours

6. Ellipse Regression

Using least squares regression, we fit an ellipse to the point cloud of contour points. To handle upper eyelid occlusion, we duplicate and reflect points from the lower half (below the y-mean), biasing the fit toward the more visible bottom contour.

Let the original contour points be:

$$P = \{(x_i, y_i) \text{ for } i = 1 \text{ to } N\}$$

First, calculate the average y-value (vertical midpoint):

$$\bar{y} = \frac{1}{N} \sum y_i$$

Then, select points below this line:

$$P_{\text{bottom}} = \{(x_i, y_i) \in P \mid y_i > \bar{y}\}$$

Reflect each of these bottom points across the horizontal line $y = \bar{y}$:

$$(x_i, y_i) \rightarrow (x_i, 2\bar{y} - y_i)$$

Finally, combine the original and reflected points:

$$P' = P \cup \text{reflected } P_{\text{bottom}}$$

This new set P' biases the ellipse fit toward the more reliable lower contour.

Ellipse Fitting Equation

We fit an ellipse in the general conic form:

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0$$

The parameters are chosen to minimize the algebraic error over all points:

$$\sum (Ax_i^2 + Bx_iy_i + Cy_i^2 + Dx_i + Ey_i + F)^2$$

With the constraint:

$$B^2 - 4AC < 0$$

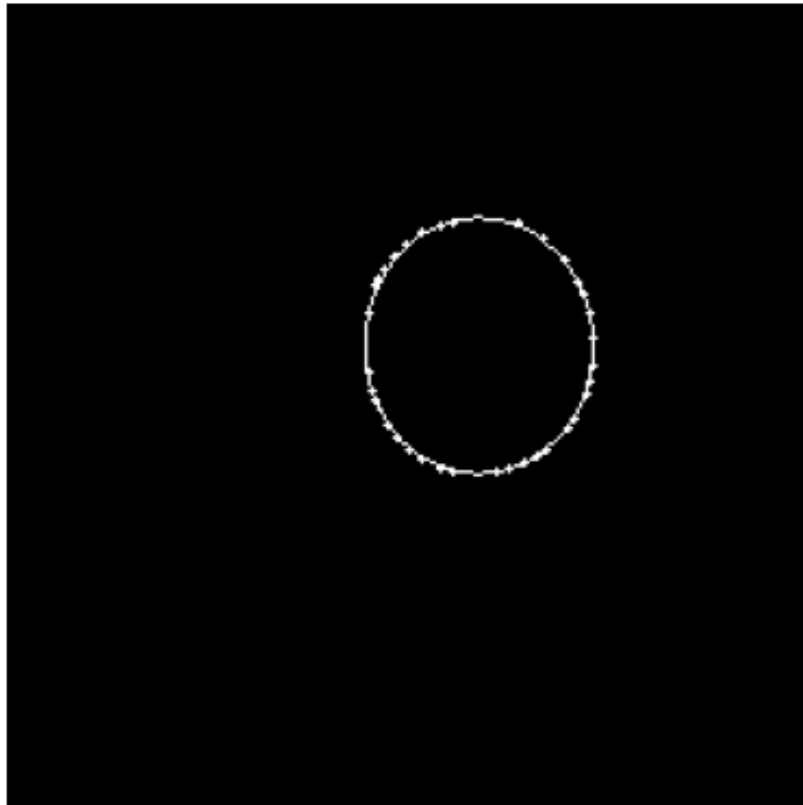


Figure 6: Ellipse Fit

7. Ellipse Selection

To choose the best ellipse, we score each candidate based on how well it fits the thresholded pupil image. The score is based on two main components:

1. **Inside White Ratio** Measures how many white pixels (likely pupil) are inside the ellipse:

$$\text{Inside Ratio} = \frac{\text{White pixels inside ellipse}}{\text{Total pixels inside ellipse}}$$

2. **Outside Black Ratio** Measures how many black pixels (non-pupil) are outside the ellipse:

$$\text{Outside Ratio} = \frac{\text{Black pixels outside ellipse}}{\text{Total pixels outside ellipse}}$$

3. **Final Score (Heuristic)** The final score favors ellipses that contain the pupil region while excluding the background:

$$\text{Score} = \text{Inside Ratio} + 0.25 \times \text{Outside Ratio}$$

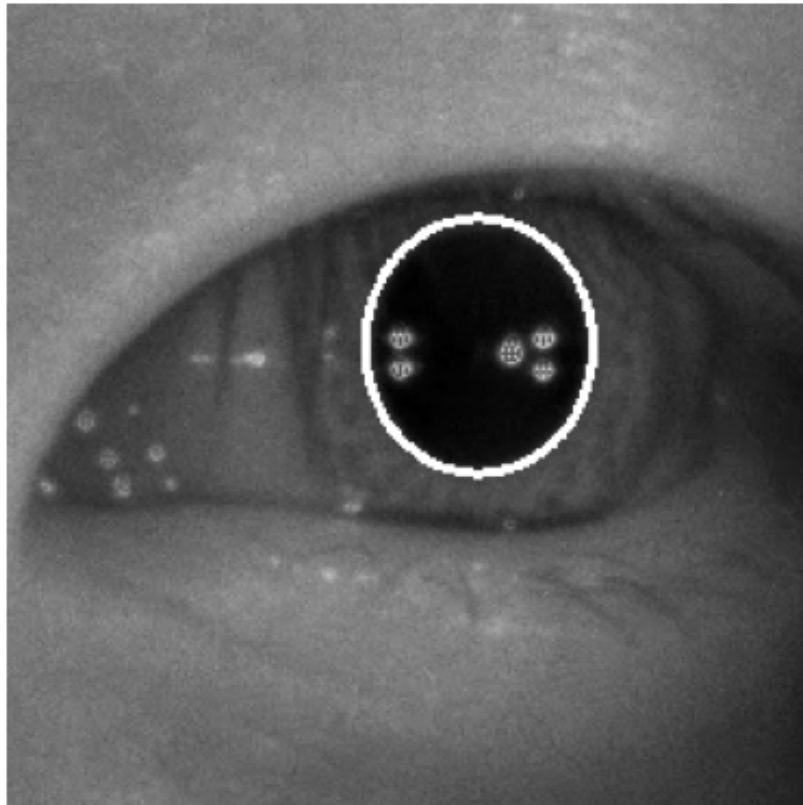


Figure 7: Ellipse Fit

8. Temporal Smoothing

To ensure smooth and consistent tracking across frames, the algorithm applies two main techniques:

1. **Change Rejection** If the newly detected ellipse differs too much from the previous one – in terms of:

- **Position** (center x , y)
- **Size** (width, height)
- **Aspect ratio** or rotation

– it is discarded and the previous ellipse is reused.

2. **Exponential Moving Average (EMA)** To make the ellipse tracking smoother and more stable, we apply a **moving average** that gives more importance to recent frames

Let: - E_t be the ellipse parameters at frame t - α be the smoothing factor (between 0 and 1)
- EMA_t be the smoothed value at frame t

We update the smoothed ellipse like this:

$$EMA_t = \alpha \cdot E_t + (1 - \alpha) \cdot EMA_{t-1}$$

Over time, this builds a weighted average like:

$$EMA_t = \alpha E_t + \alpha(1 - \alpha)E_{t-1} + \alpha(1 - \alpha)^2 E_{t-2} + \dots$$

This helps reduce jitter, absorb small errors, and maintain temporal consistency.

3 CNN Overview

We aim to train a Convolutional Neural Network (CNN) to segment the pupil directly from the image, avoiding classical image processing techniques. This approach improves **portability**, **stability**, and **generalization**, while also allowing **data augmentation** to simulate challenging real-world scenarios.

Given an input image (left), the network should produce a segmentation mask (right):

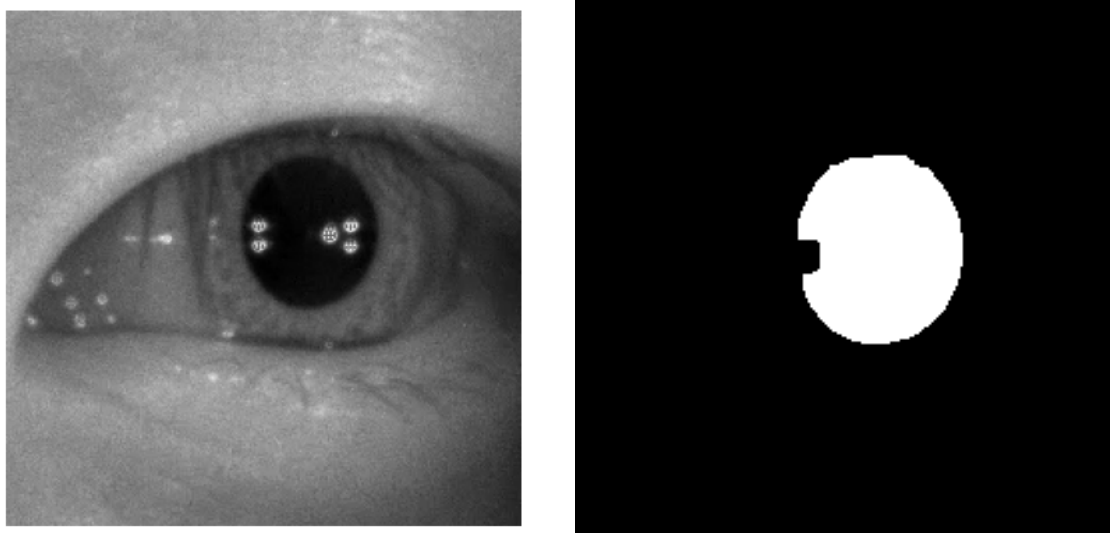


Figure 8: Eye Crop Input and Output

4 Data Preparation Augmentation

The script `video2frames.py` extracts frames from videos located in the `videos/` folder and saves them to the `frames/` directory.

Then, `generate_masks.py` processes each frame to generate:

- An **input image** (saved to `images/`)
- A **corresponding segmentation mask** (saved to `best_masks/`)

Each image-mask pair is then duplicated and augmented using various transformations, including:

- Brightness adjustment
- Contrast enhancement
- Sharpness modification
- Gaussian blur
- Gamma correction
- Rotation
- Stretching

5 CNN Training

A scaled down SegNet model is used.

Training is done in `CNN.ipynb`

- **Input:** A 128×128 grayscale image, represented as a matrix of pixel values in the range $[0, 255]$.
- **Shape:** (128, 128, 1) — height, width, and a single channel.
- **Normalization:** The pixel values are scaled to the range $[0, 1]$ to improve training stability and convergence during model optimization.

The model is trained using **Binary Cross-Entropy loss**, which compares each predicted pixel value to the ground truth label.

Let: - $y_i \in \{0, 1\}$ be the ground truth label for pixel i - $\hat{y}_i \in (0, 1)$ be the predicted probability for pixel i - N be the total number of pixels in the mask

Then the Binary Cross-Entropy loss is:

$$\text{BCE Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

This loss penalizes incorrect predictions more strongly when the model is confident but wrong. It's ideal for binary segmentation tasks like pupil detection.

After 15 epochs with a batch size of 32 using Adam for optimization, we achieve 99

The model follows a basic **encoder-decoder architecture**:

Encoder (Feature Compression)

Reduces spatial dimensions while increasing the number of filters to extract rich features.

```
x = Conv2D(32, 3, padding="same", activation="relu")(inputs)
x = MaxPooling2D(2)(x)

x = Conv2D(64, 3, padding="same", activation="relu")(x)
x = MaxPooling2D(2)(x)

x = Conv2D(128, 3, padding="same", activation="relu")(x)
x = MaxPooling2D(2)(x)
```

Decoder (Upsampling)

Restores the original image resolution by upsampling and refining features.

```
x = UpSampling2D(2)(x)
x = Conv2D(64, 3, padding="same", activation="relu")(x)

x = UpSampling2D(2)(x)
x = Conv2D(32, 3, padding="same", activation="relu")(x)

x = UpSampling2D(2)(x)
```

Output Layer

```
outputs = Conv2D(1, 1, activation="sigmoid")(x)
```

- Produces a **128×128 mask** where each pixel value is between 0 and 1. - Represents the **probability of each pixel being part of the pupil**. - Values with a confidence above 0.5 are interpreted as the pupil

6 Inference

The trained model is exported to the **ONNX format** for portability and cross-platform compatibility.

- A frame is captured from the video stream.
- The eye region is localized and cropped using the same logic as in the original algorithm.
- The cropped grayscale image is preprocessed and passed into the CNN.
- The model outputs a **binary segmentation mask** predicting the pupil region.
- This mask is post-processed, and a **single ellipse** is fitted to the largest detected contour in the mask.
- The approach combines the speed and simplicity of a learned model with the precision of geometric post-processing, running at 60 FPS in testONNX.py.

7 Additional Notes

- CNN to directly fit ellipse is costly and innaccurate
- Program performs better on CPU's with high clock speeds
- Can try different architectures of model
- SGD could be used for better generalization but requires more hyperparameter tuning